# FIBONACCI SERIES IN DATA STRUCTURES

# CASE STUDY ASSIGNMENT


**GROUP MEMBERS**

- GIDEON KIPROP CT204/125696/25
- ROZE WAITHIRA CT204/124287/25
- SHEILA CHERAOTICH CT204/124322/25
- ENOCK BOSIRE CT204/124361/25

1. INTRODUCTION ..... 4
1.1 Objectives ..... 4
1.2 Reward-point convention ..... 4
2. TASK 1: RECURSIVE APPROACH ..... 4
2.1 Explanation ..... 4
2.2 C++ program code ..... 5
2.3 Complexity analysis ..... 5
2.4 Advantages ..... 5
2.5 Disadvantages ..... 5
TASK 2: ITERATIVE APPROACH ..... 6
3.1 Explanation ..... 6
3.2 C++ implementation ..... 6
3.3 Complexity analysis ..... 7
3.4 Advantages ..... 7
3.5 Disadvantages ..... 7
TASK 3: DYNAMIC PROGRAMMING USING TABULATION ..... 7
4.1 Explanation of tabulation ..... 7
4.2 C++ implementation ..... 7
4.4 Complexity and space-time trade-off ..... 8
4.5 Advantages ..... 8
4.6 Disadvantages ..... 9
TASK 4: CUSTOMER LOYALTY REWARD SYSTEM ..... 9
5.1 Problem model ..... 9
5.2 C++ implementation ..... 9
5.3 Sample output ..... 10
5.4 Explanation of the application ..... 10
TASK 5: PERFORMANCE ANALYSIS AND RECOMMENDATION ..... 11

5.1 Performance Comparison of Fibonacci Algorithms ..... 11

5.2 Comparison of the Three Approaches ..... 11

5.3 Performance When Calculating the 30th Fibonacci Number ..... 11

5.4 Recommended Approach ..... 12

6. CONCLUSION ..... 12

7. REFERENCES ..... 13

# 1. INTRODUCTION

The Fibonacci sequence is a series of numbers in which each number, after the first two, is obtained by adding the two preceding numbers.

In this case study, a software company wants to use the Fibonacci sequence to determine customer loyalty reward points. The number of years a customer remains loyal is entered and the program displays the reward points according to each year. Then the study implements and compares three approaches that is recursion, iteration and dynamic programming using tabulation.

## 1.1 Objectives

- To use a recursive function that computes the position of a Fibonacci number.
- To generate and store Fibonacci values in an array.
- To apply dynamic programming through tabulation and analyze its time-space trade-off.
- To develop a customer loyalty system that will display the reward points for each year.
- To compare execution times at n = 30 and recommend a suitable approach for the company.

## 1.2 Reward-point convention

In the Reward system, Year 1 is assigned no point, year 2 is assigned 1 point, and each subsequent year follows the Fibonacci sequence.

# 2. TASK 1: RECURSIVE APPROACH

## 2.1 Explanation

Recursion is a programming technique in which a function calls itself to solve smaller instances of the same problem.

The recursive approach repeatedly computes the same values. For example, calculating F(5) requires F(4) and F(3), while the calculation of F(4) also requires F(3). This duplication makes the approach inefficient as the no of n increases.

## 2.2 C++ program code

```
#include <iostream>
using namespace std;

long long fibRecursive(int n)
{
    if (n <= 1)
    {
        return n;
    }

    return fibRecursive(n - 1)
         + fibRecursive(n - 2);
}

int main()
{
    int n;

    cout << "Enter the Fibonacci position\n";
    cin >> n;

    if (cin.fail() || n < 0)
    {
        cout << "Please enter a non-negative integer." << endl;
        return 1;
    }

    cout << "Fibonacci number at position " << n
         << " is " << fibRecursive(n) << endl;

    return 0;
}
```

## 2.3 Complexity analysis

Time complexity: O(2^n) is a valid upper bound for the basic recursive algorithm. The running time grows exponentially because many subproblems are recalculated.

Space complexity: O(n), because recursive calls are held on the call stack and the maximum call depth grows linearly with n.

## 2.4 Advantages

- The code is simple to understand.
- It closely follows the mathematical definition.
- It demonstrates how recursive functions work.

## 2.5 Disadvantages

- It recalculates the same Fibonacci numbers repeatedly.

- Its execution time increases rapidly for larger inputs.
- Deep recursion can exhaust the call stack.

Due to very many repeated calculations as the n grows exponentially, this makes it inefficient for large inputs.

# TASK 2: ITERATIVE APPROACH

## 3.1 Explanation

Iteration repeats instructions using a loop. The first two values are initialized, and each later value is calculated by adding the two preceding array entries. This avoids the repeated calculations made by basic recursion.

## 3.2 C++ implementation

```
#include <iostream>
#include <vector>
using namespace std;

vector<long long> fiboIterative(int n)
{
    vector<long long> fib(n + 1, 0);

    if (n >= 1)
    {
        fib[1] = 1;
    }

    for (int i = 2; i <= n; i++)
    {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib;
}

int main()
{
    int n;

    cout << "Enter the highest Fibonacci position: ";
    cin >> n;

    if (cin.fail() || n < 0)
    {
        cout << "Please enter a non-negative integer." << endl;
        return 1;
    }

    vector<long long> fib = fiboIterative(n);

    cout << "Fibonacci series: ";
    for (int i = 0; i <= n; i++)
    {
        cout << fib[i] << " ";
```

```
    }
    cout << endl;

    return 0;
}
```

### 3.3 Complexity analysis

Time complexity: O(n), because the loop calculates each value from index 2 through n once.

Space complexity: O(n), because the array stores n + 1 values, including F(0).

### 3.4 Advantages

- Faster than basic recursive computation for larger inputs.
- Avoids repeated recursive calls.
- Stores the complete sequence for later use.
- Makes it easy to display the reward points for each year.

### 3.5 Disadvantages

- Requires additional memory to store the sequence.
- The array grows as the requested number of years increases.

Iteration uses loop to calculates each Fibonacci number once e.g generating the sequence up to position 30 requires approximately 29 additions after initializing the first two values as compared this with recursion, which performs millions of calls for a similar input.

## TASK 3: DYNAMIC PROGRAMMING USING TABULATION

### 4.1 Explanation of tabulation

Dynamic programming solves a problem by storing solutions to smaller subproblems and reusing them. Tabulation is the bottom-up form of dynamic programming: it starts with known base cases and fills a table in order until the required result is available.It does not rely on recursive calls

### 4.2 C++ implementation

```
#include <iostream>
```

```
#include <vector>
using namespace std;

long long fiboTabulation(int n)
{
    vector<long long> fib(n + 1, 0);

    if (n >= 1)
    {
        fib[1] = 1;
    }

    for (int i = 2; i <= n; i++)
    {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib[n];
}

int main()
{
    int n;

    cout << "Enter the Fibonacci position: ";
    cin >> n;

    if (cin.fail() || n < 0)
    {
        cout << "Please enter a non-negative integer." << endl;
        return 1;
    }

    cout << "Fibonacci number at position " << n
         << " is " << fiboTabulation(n) << endl;

    return 0;
}
```

## 4.4 Complexity and space-time trade-off

Time complexity: O(n), because each table entry is calculated once. The loop calculates each Fibonacci value once. Therefore, the number of calculations grows linearly with the input.

Space complexity: O(n), because the table stores n + 1 Fibonacci values. Tabulation uses additional memory to avoid recalculation and makes the intermediate results available for later use. Although this requires additional memory, the stored values eliminate repeated calculations and can be accessed whenever needed.

## 4.5 Advantages

- Avoids the repeated calculations associated with basic recursion.
- Has linear time complexity.
- Does not require recursive function calls.
- stores intermediate results for later use.

- ❖ Builds the solution in a predictable, bottom-up order.

### 4.6 Disadvantages

- ➢ Requires memory for the table.
- ➢ Calculates all preceding Fibonacci values even when only the final value is required.
- ➢ Must allocate sufficient space for the requested input.

Note: The iterative array method in Task 2 and tabulation in Task 3 can have nearly identical code and the same O(n) time and space complexities. Tabulation is presented separately to demonstrate the dynamic programming principle of storing and reusing solutions to subproblems; it should not be claimed to be automatically faster than iteration.

## TASK 4: CUSTOMER LOYALTY REWARD SYSTEM

### 5.1 Problem model

Input the number of complete years a customer has remained loyal then generate Fibonacci reward points for years 1 through n. Keep the rewards in a vector and finally display the reward points according to each year.

### 5.2 C++ implementation

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    int years;

    cout << "CUSTOMER LOYALTY REWARD SYSTEM" << endl;
    cout << "Enter the number of loyal years: ";
    cin >> years;

    if (cin.fail() || years < 1)
    {
        cout << "Invalid input. Enter a positive number of years."
             << endl;
        return 1;
    }

    vector<long long> rewards(years);

    rewards[0] = 0;
```

```
    if (years >= 2)
    {
        rewards[1] = 1;
    }

    for (int i = 2; i < years; i++)
    {
        rewards[i] = rewards[i - 1] + rewards[i - 2];
    }

    cout << "\nCustomer Reward Summary" << endl;


    for (int i = 0; i < years; i++)
    {
        cout << "Year " << i + 1 << ": "
             << rewards[i] << " reward points" << endl;
    }

    return 0;
}
```

## 5.3 Sample output

```
CUSTOMER LOYALTY REWARD SYSTEM
Enter the number of loyal years: 8

CUSTOMER REWARD SUMMARY
Year 1: 0 reward points
Year 2: 1 reward points
Year 3: 1 reward points
Year 4: 2 reward points
Year 5: 3 reward points
Year 6: 5 reward points
Year 7: 8 reward points
Year 8: 13 reward points
```

## 5.4 Explanation of the application

1. Input validation checks that the user enters a positive integer.
2. A vector named rewards is created with one element for each loyalty year.
3. The first two years are initialized to zero point and one point respectively, when those years are included.
4. A loop calculates each later reward by adding the two preceding rewards.
5. A final loop displays each year and its corresponding reward points.

The program demonstrates the algorithm rather than a complete production customer-management system.

# TASK 5: PERFORMANCE ANALYSIS AND RECOMMENDATION

## 5.1 Performance Comparison of Fibonacci Algorithms

## 5.2 Comparison of the Three Approaches

| Approach | Time Complexity | Space Complexity | Performance |
|---|---|---|---|
| Recursive | O(2^n) | O(n) | Slow for large inputs because it repeatedly calculates the same Fibonacci numbers. |
| Iterative | O(n) | O(1) when only two previous values are stored | Efficient because it calculates each Fibonacci number once and uses very little additional memory. |
| Dynamic Programming (Tabulation) | O(n) | O(n) when an array stores all values | Efficient because it calculates each Fibonacci number once and stores the results for later use. |

## 5.3 Performance When Calculating the 30th Fibonacci Number

Using the standard Fibonacci sequence, where F(0)=0 and F(1)=1, the 30th Fibonacci number is:

F(30)=832040

The three approaches produce the same result, but their performance differs.

- **Recursive approach:** This approach is the slowest because it repeatedly calculates the same Fibonacci values. As the input increases, the number of function calls grows rapidly, making the algorithm inefficient for larger inputs.
- **Iterative approach:** This method is faster because it calculates the Fibonacci numbers sequentially, using the previously calculated values thus avoids unnecessary repeated calculations and requires very little additional memory when only two previous values are stored.
- **Dynamic programming using tabulation:** This approach is also efficient because it calculates each Fibonacci number once and stores the results in an array. This makes previously calculated values available whenever they are needed.

Although the iterative and tabulation approaches both have linear time complexity, their memory requirements differ when tabulation stores the complete sequence in an array.

### 5.4 Recommended Approach

Based on the analysis of the three approaches, we recommend dynamic programming using tabulation for the customer loyalty reward system. Why?

First, tabulation has a time complexity of O(n), making it more efficient than basic recursion, which has exponential time complexity. It calculates each Fibonacci number once and stores the result in an array, avoiding unnecessary repeated calculations.

Second, tabulation does not require recursive function calls, reducing function-call overhead and avoiding the risk of excessive recursion depth. It also stores the reward points for each year, enabling the system to retrieve and display a customer's reward history efficiently.

Finally, tabulation is suitable for future expansion because its running time grows linearly with the number of years being processed. Although the iterative approach also provides linear time complexity and similar performance, tabulation explicitly demonstrates the dynamic programming technique of building solutions from previously calculated subproblems.

In conclusion, dynamic programming using tabulation is the recommended approach because it combines computational efficiency, systematic storage of intermediate results, and suitability for generating the complete reward sequence. These characteristics make it a better choice for the company's customer loyalty reward system.

We recommend tabulation because it explicitly applies dynamic programming by storing solutions to smaller subproblems and using them to calculate larger ones. Since the system needs to generate and store the reward points for every year, tabulation is a suitable approach. Its advantage over ordinary iteration is primarily the dynamic programming methodology rather than necessarily faster execution.

## 6. CONCLUSION

This case study examined three ways of computing Fibonacci numbers for a customer loyalty reward system. Basic recursion is straightforward and reflects the mathematical definition, but it repeats calculations and has exponential running time. Iteration with an array computes the sequence in linear time and stores the results. Dynamic programming using tabulation also has linear time complexity and builds a table from smaller subproblems to larger ones.

For the customer reward system, tabulation is recommended because it efficiently generates and stores the complete sequence of yearly reward points without recursive-call overhead.

## 7. REFERENCES


- Course lecture notes on data structures and algorithm analysis